

Contents

1 DRAFT web scraping	1
1.1 what is web scraping?	1
1.1.1 from bytes to HTML	1
1.1.2 HTTP	2
1.1.3 Blockers and the resistance	5
1.2 bypassing blockers: scraping congress.gov	6

1 DRAFT web scraping

- add scraping of heritage foundation website

1.1 what is web scraping?

1.1.1 from bytes to HTML

Web scraping is the extraction of information from websites. It most often involves (with some exceptions, explained below) working with data in HTML format, with elements and attributes.

```
<!DOCTYPE html>
<html>
<body>

<h1 class="header">First level heading element</h1>

<p>A paragraph element with a <a href="/">link</a> inside of it</p>

</body>
</html>
```

Websites are stored in web servers, accessed over a network. Web data travels over the network in the form of bytes, the smallest digitally addressable data unit. When the bytes reach the web browser, they are re-encoded into HTML language, which is assembled into a web page readable by human eyes.

1.1.2 HTTP

Web *scraping* is possible by manipulating HTTP, or HyperText Transfer Protocol, which is the language of the network. A web scraper uses code to manipulate an HTTP request so that it receives the website data in the form of code.

Web scraping is, at its core, about mimicking a typical browser request. Using HTTP protocols, it makes a request to a web server much like a user makes a request by typing in a URL into the browser. The data it gets back from the HTTP request is also the same as the data that comes to a browser after searching a URL—only the data is intercepted before in the form of bytes, rather than assembled into a web page visible through a browser.

```
r = requests.get("https://www.nytimes.com/")
```

You can scrape a website with only five lines of code. Below is an example that scraps the home page of *The New York Times* website using the `bs4` and `requests` libraries in python.

```
from bs4 import BeautifulSoup
import requests

r = requests.get("https://www.nytimes.com/")
source = r.content
soup = BeautifulSoup(source, 'html.parser')
```

The `requests` library is one of the most popular libraries in the python programming language. Enabling programs to send and receive data through HTTP networks (i.e., the internet), it is the bedrock for virtually all web scraping and API tasks. The `get()` function from `requests` can access the *NYTimes* homepage, which is saved in the form of bytes via the `content` property. Then, using `bs4`, a python library for working with HTML data, the data is parsed into an object called `soup` (following `bs4`'s naming conventions), which is a pythonic object of HTML data.

The beauty of `bs4` is that it makes HTML—a notoriously fussy language that is the reason why so many web frameworks exist in the first place, attempting to abstract the user from working directly in HTML—accessible through simple python syntax.

For example, the expression `soup.title` returns the following output:

```
>>> soup.title
<title data-rh="true">The New York Times - Breaking News, US News, World News and Video
```

Similarly, other HTML elements can now be accessed using python dot syntax.

```
>>> soup.p
<p>Make sense of the day's news and ideas.</p>

>>> soup.a
<a class="css-kgn7zc" href="#site-content">Skip to content</a>

>>> soup.h1
<h1 class="css-1dv1kvn">New York Times - Top Stories</h1>
```

Further properties like `text` (for just text) and functions like `find()` are also possible. Functions can be further specified when combined with the `class_` parameter, to search the HTML with more precision. Due to python syntax, they can also be chained together to create complex expressions. These building blocks enable programmers to navigate the HTML tree with granularity.

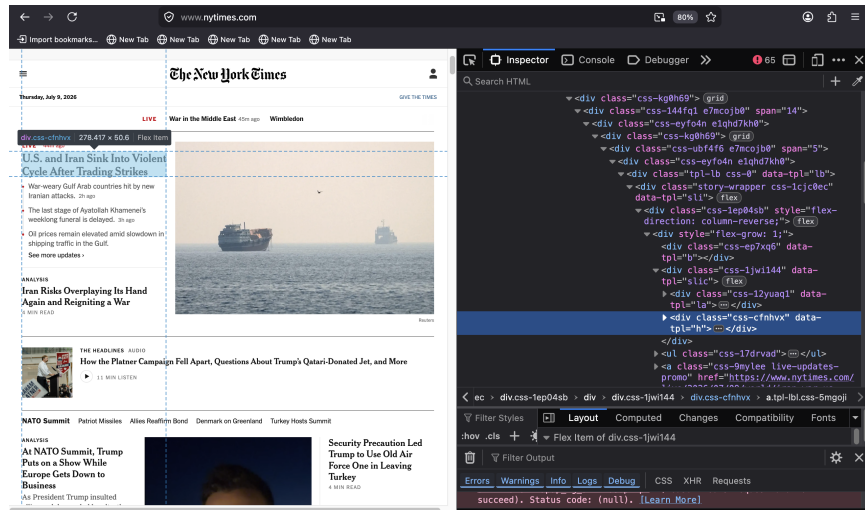
```
>>> soup.h1.text
'New York Times - Top Stories'

>>> soup.find('div', class_ = 'css-cfnhvx')
<div class="css-cfnhvx" data-tpl="h"><a class="tpl-lbl css-5mgoji" data-tpl="l" href="l">U.S. and Iran Sink Into Violent Cycle After Trading Strikes'

>>> soup.find('div', class_ = 'css-cfnhvx').text
'U.S. and Iran Sink Into Violent Cycle After Trading Strikes'
```

HTML classes for an element can be found by using built-in browser tools like the web inspector. On the webpage, right click on the relevant text and select **inspect** from the dropdown menu. The element will be highlighted with information for the `class` attribute.¹

¹Note that across programming languages like HTML and python, terms like "attributes" mean different things and in practice function very differently. In HTML, an attribute is data contained within an HTML element, like a value specifying a CSS class, such as `<h1 class=title>New York Times</h1>`. By contrast, in python an attribute is information associated with an object model, like `r.content` or `r.status_code`, which is data associated with an HTTP request object, `r`.



Using requests, bs4, and the web inspector, one can easily grab a list of the current headlines from the *NYTimes* homepage.

```
>>> for i in soup.find_all('div', class_ = 'css-cfnhvx'):
    print(i.text)
```

U.S. and Iran Sink Into Violent Cycle After Trading Strikes

Iran Risks Overplaying Its Hand Again and Reigniting a War

At NATO Summit, Trump Puts on a Show While Europe Gets Down to Business

Security Precaution Led Trump to Use Old Air Force One in Leaving Turkey

'A Slow-Rolling Disaster': Inside the Implosion of the Platner Campaign

What's Next for Democrats in the Senate Race in Maine

How Democrats and Republicans Diverge on Sexual Assault Allegations

Platner Suspends Senate Bid in Maine After Rape Accusation

Trump's Loyal Defender at the Vatican

Trump Says He'll Ask Supreme Court to Rehear Citizenship Case, an Unlikely Event

Can Trump's Arch Be So Tall? A Law May Be Redefined to Get to Yes.

Morocco Faces France and Its Star, Mbappé, in Quarterfinals

A Colonial Past Mixes With the Global Present

How Morocco's Diaspora Became the Team's Driving Force

13 Red Cards Have Been Given at the World Cup. Not All Have Been Equal.

Anything else to say about HTTP?

1.1.3 Blockers and the resistance

Web servers, unfortunately for the scraper, often block automated requests to their data. They use various forms of "blockers" or mechanisms that deny the HTTP request, sending back a code 403, or "Forbidden" message.

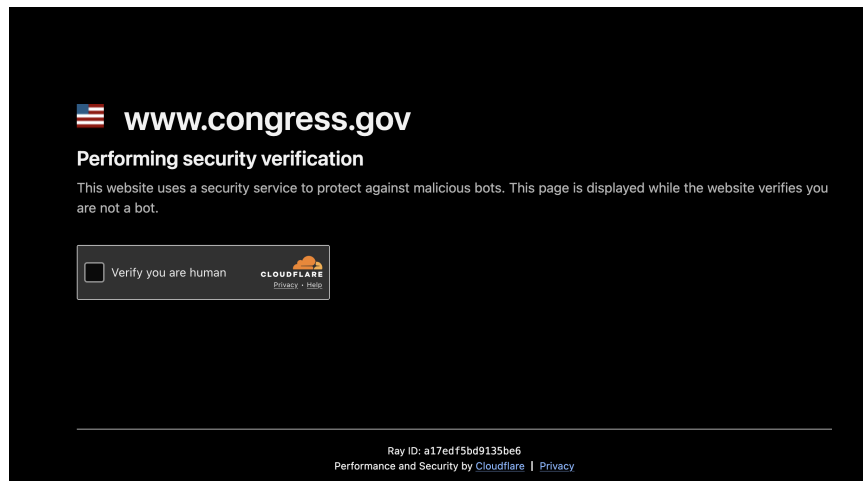
Each year of the past few years, bot traffic has increased exponentially, coming mostly from major tech and AI companies, especially OpenAI, META, Google, and Anthropic. According to Cloudflare, 2025 saw increases of 19% in bot traffic (See Fig #) (Cloudflare, "From Googlebot to GPTBot: who's crawling your site in 2025"). It is no surprise that traffic has increased mostly from these companies, whose ML model training requires ever increasing amounts of data.

Rank	Bot Name	Share (May 2024)	Rank	Bot Name	Share (May 2025)
1	Bytespider	42%	1	GPTBot	30%
2	ClaudeBot	27%	2	ClaudeBot	21%
3	Amazonbot	21%	3	Meta-ExternalAgent	19%
4	GPTBot	5%	4	Amazonbot	11%
5	Applebot	4.1%	5	Bytespider	7.2%

As automated scrapers increase in frequency, so do the bot blockers. Over the past two years, Cloudflare, which runs on 20% of total internet pages and 50% of pages based in the US, has added increasing bot detection like "Challenges" and "Turnstiles" to its web pages. Some of these operate under the surface and virtually invisible to the user, such as checking the HTTP request metadata like the "User-Agent." By default, the User-Agent is set to the name of web browser and operating system making the request. For example, my Firefox browser running Mac OSX has the User-Agent `Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:152.0) Gecko/20100101 Firefox/152.0`. The python `requests` library, by contrast, sets the User-Agent to `python-requests/2.32.3`, making any HTTP request using that library easy for blockers to detect.

Beyond examining HTTP request data, bot blockers will use JavaScript code to check if the request is from a real web browser, rather than from an automated program. JavaScript, which is a programming language native to web browsers (and discussed in more detail in Chapter 4: Text Display), can inject challenges for the browser to solve, challenges which may or may not involve the user. All of these challenges might require some processing from the browser, and increasingly, they may require the user's interaction (to

confirm that the request comes from a real browser, not a bot). One popular challenge is the "security verification" checkbox, which is increasingly visible across the internet (See Fig #).



While these kinds of blockers are more advanced, there are ways of overcoming them. One effective method involves using a virtual or "ghost" browser through python libraries like `selenium`. Instead of sending an HTTP request via a python script, `selenium` routes the request through an actual web browser—though one that is operated remotely by the python script. When the program is running, a browser window will open on its own, navigate to a web page, and click on links, all without the input of the user. It looks like a "ghost" is operating the computer. One major drawback to this method is that it requires a virtual display, which many "headless" scraping servers do not have, in order to access the data via the browser window. That requirement, in addition to the increase in necessary processing power, detracts large-scale web crawling.

1.2 bypassing blockers: scraping congress.gov

- incorporate this section into the blockers conversation

Scraping the congress.gov servers by constructing a URL using meta-data from congressional records. Manipulating the URL to circumventing the protections on the servers for accessing the records.

- the protections on congress.gov

- how to construct a URL
- the dataset - congressional records on anti-trans legislation
- analysis of the dataset? close reading?